

🌱 Beginner-Friendly EKS + AWS Service Challenge

This challenge helps you get comfortable with **AWS EKS** and connecting your containerized app to **one additional AWS service**.

🎯 Objective

Create a basic "Hello from EKS!" web app in any language of your choice and integrate it with a **single AWS service** (choose one):

- 📁 **S3** - Read/write a simple file
- 📧 **SQS** - Send a message to a queue
- 📖 **DynamoDB** - Fetch a sample record
- 🔑 **Secrets Manager** - Read a secret at runtime

You'll:

- Deploy the app to an existing EKS cluster
- Use a Kubernetes service account mapped to an IAM role (IRSA)
- Inject configuration via environment variables

✂️ Tech Stack (Pick your own)

✅ Supported languages (choose any you're comfortable with):

- Python (Flask)
- JavaScript (Node.js + Express)
- Java (Spring Boot)
- C# (.NET Core)

💡 You only need to connect to **one AWS service**.

🌿 What You Need To Do

1. **Build a basic app** (any language) that shows:

```
```\nHello from EKS! Fetched <S3/DynamoDB/Secret> successfully!\n```
```

2. **Package the app using Docker**

3. **Create a Kubernetes manifest (Pod/Deployment)** that:

- Uses a **service account** linked to an **IAM role (IRSA)**
- Injects config (like S3 bucket name, secret ID) using environment variables

4. **\*\*Deploy the Pod to EKS\*\*** using:

```
```bash
kubectl apply -f your-app.yaml
```
```

---

## 📖 Reference: IRSA (IAM Roles for Service Account)

Use IRSA to securely give your pod permission to access S3/DynamoDB/Secrets without hardcoding credentials.

📖 [IRSA Setup Guide](https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html)

---

## 💡 Helpful Examples

- S3 with Python: <https://boto3.amazonaws.com/v1/documentation/api/latest/guide/s3-example-creating-buckets.html>
- IRSA in EKS: <https://aws.github.io/aws-eks-best-practices/iam/iam-roles-for-service-accounts/>
- Secrets Manager usage in Python: <https://docs.aws.amazon.com/secretsmanager/latest/userguide/retrieving-secrets.html>

---

## ✅ Expected Deliverables

- Your app code
- Dockerfile
- Kubernetes manifest (`pod.yaml` or `deployment.yaml`)
- Screenshot or terminal output proving integration worked

---

## 🎁 Bonus (Optional)

- Expose using LoadBalancer
- Use ConfigMap or Secret instead of hardcoding env vars
- Try using a Helm chart

---

Have fun exploring real AWS integrations, one step at a time! 🌱 ☁️